SafeBank : plan des sprints

Date limite de rendu : dimanche 27 décembre 2026. Objectif du binôme : projet terminé le mardi 15 décembre 2026, date du dernier cours de Programmation distribuée. Du 16 au 27 décembre, on ne garde que la marge : corrections, rapport final et envoi.

M = Mamadou, **B** = binôme, **M+B** = ensemble. Chaque tâche se termine par une Pull Request relue par l'autre. Une tâche est « finie » quand : le code est sur `main`, les tests passent, et une capture d'écran est rangée dans `docs/captures/` si elle sert au rapport.

Repère	Date
Cours Programmation distribuée	9/11, 14/12, 15/12 (matin)
Cours Conteneurisation	18/11, 23/11, 2/12
Cours Cloud Computing	16/11, 30/11, 7/12
Hackathon (moins de temps pour le projet)	24-25/11
Fin visée du projet	**15/12**
Envoi au prof, au plus tard	**27/12** (viser le 20/12, avant les vacances de Noël)

Sprint 0 · Mise en place (14 → 20 octobre)

Objectif : un dépôt propre, une structure en couches, tout le monde sait lancer le projet.

#	Tâche	Qui
0.1	Ajouter le binôme comme collaborateur du dépôt GitHub, protéger `main` (PR obligatoire)	M
0.2	Ajouter `docs/` (ces trois documents) et un `README.md` racine	M
0.3	Réorganiser `account-service` en couches : `entity/`, `dto/`, `repository/`, `service/`, `web/`, `exceptions.py`	M
0.4	Ajouter ruff (`uv add --dev ruff`) et le configurer dans `pyproject.toml`	M
0.5	Créer le squelette `services/fraud-service/` (uv, structure en couches, `GET /health`)	B
0.6	Installer en local : Docker, minikube, kubectl ; créer un compte Docker Hub chacun	M+B

Sprint 1 · Palier 10/20 : un service complet dans Kubernetes (21 octobre → 3 novembre)

#	Tâche	Qui
1.1	Entité `Transfer` + `TransferStatus` (`ACCEPTED`, `PENDING_REVIEW`, `REJECTED`)	M
1.2	`TransferService` : règles 1 à 4 (comptes existants, montant > 0, comptes différents, solde suffisant)	M
1.3	Routes `POST /transfers` et `GET /accounts/{id}/transfers` ; traduction des exceptions métier en codes HTTP	M
1.4	Tests unitaires de la couche service (avec un faux repository) + tests API	M
1.5	`Dockerfile` + `.dockerignore` de `account-service` (non root, uv)	M
1.6	Publier l'image `artmis137/safebank-account-service:0.1.0` sur Docker Hub	M
1.7	`k8s/deployment.yaml` (2 réplicas, probes sur `/health`) et `k8s/service.yaml` ; test sur minikube (`kubectl port-forward`)	M
1.8	`fraud-service` : entité, règles de score (tableau du doc projet), `POST /score`, tests	B
1.9	Workflow GitHub Actions par service (ruff + pytest + build Docker), déclenché seulement si le dossier du service change	B

Livrable : captures de `kubectl get pods,svc`, de Docker Hub et d'un appel à l'API dans le cluster.

Sprint 2 · Paliers 12/20 et 14/20 : gateway + deuxième service relié (4 → 17 novembre)

> Cours : Programmation distribuée le 9/11, Cloud Computing le 16/11.

#	Tâche	Qui
2.1	Dockerfile + image `safebank-fraud-service:0.1.0` + Deployment/Service Kubernetes	B
2.2	Couche `client/` dans `account-service` : appel REST à `fraud-service` (URL dans `config.py`, timeout court)	M
2.3	Règle 5 (décision ALLOW / REVIEW / BLOCK) et règle 6 (fraud-service indisponible → 503) dans `TransferService` + tests	M
2.4	ConfigMap pour `FRAUD_SERVICE_URL`	M
2.5	Gateway : Ingress minikube, puis Istio Gateway + VirtualService vers `account-service`	M+B
2.6	Vérifier que `fraud-service` n'est pas joignable depuis l'extérieur du cluster	B

Livrable : capture d'un virement accepté et d'un virement bloqué passant par la gateway.

Sprint 3 · Palier 16/20 : base de données (18 → 29 novembre)

> Cours : Conteneurisation les 18 et 23/11. Hackathon les 24-25/11 : prévoir peu de travail ces deux jours.

#	Tâche	Qui
3.1	Repository PostgreSQL (SQLAlchemy 2) qui remplace le repository en mémoire, sans changer la couche service	M
3.2	Migrations Alembic (tables `accounts`, `transfers`)	M
3.3	Virement dans une **transaction** (débit + crédit + enregistrement, tout ou rien)	M
3.4	PostgreSQL dans Kubernetes : PVC + Secret + Service	B
3.5	`docker compose` pour lancer les deux services + PostgreSQL en local sans Kubernetes	B
3.6	Test : supprimer un pod `account-service`, vérifier que les données sont toujours là	M+B

Sprint 4 · Palier 18/20 + bonus gRPC (30 novembre → 8 décembre)

> Cours : Cloud Computing le 30/11 et le 7/12, Conteneurisation le 2/12.

#	Tâche	Qui
4.1	`proto/fraud.proto` (contrat du doc technique) + génération du code Python	B
4.2	Serveur gRPC dans `fraud-service` (port 50051), réutilisant la même couche service que le REST	B
4.3	Client gRPC dans `account-service/app/client/` ; choix REST/gRPC par `FRAUD_TRANSPORT`	M
4.4	Istio : mTLS strict dans le namespace `safebank` (PeerAuthentication)	M+B
4.5	RBAC : ServiceAccount dédié par service, Role/RoleBinding avec le minimum de droits	M
4.6	Secrets Kubernetes pour toutes les données sensibles ; rien de sensible dans Git	M
4.7	Sécurité Docker Hub : 2FA, jeton d'accès pour la CI au lieu du mot de passe ; scan des images (Trivy) dans la CI	B

Sprint 5 · Palier 20/20 : cloud + rapport (9 → 15 décembre)

> Cours : Programmation distribuée les 14 et 15/12. **Le 15/12, le projet doit tourner de bout en bout.**

#	Tâche	Qui
5.1	Cluster GKE sur Google Cloud (crédits d'essai ou projet fourni en cours), déploiement des deux services + PostgreSQL	M+B
5.2	Vérifier tout le parcours sur GKE : gateway → account-service → fraud-service (gRPC) → PostgreSQL	M+B
5.3	**Rapport, première version** : architecture, une section par palier avec ses captures, mode d'emploi	M+B
5.4	Rassembler les captures individuelles des Google labs de chacun	M+B
5.5	Profiter des cours des 14 et 15/12 pour montrer le projet au prof et poser les dernières questions	M+B

Marge · Finitions et envoi (16 → 27 décembre)

> Vacances de Noël à partir du 21/12 : viser un envoi le **20/12**, au plus tard le **27/12**.

#	Tâche	Qui
6.1	Corriger ce qui reste après les retours du 14-15/12	M+B
6.2	Rapport final relu par les deux, `README` à jour, tag `v1.0.0`	M+B
6.3	Envoi par mail à benoit.charroux@gmail.com : rapport + lien du dépôt + captures Google labs	M
6.4	Seulement s'il reste du temps : front React minimal, ou modèle scikit-learn à la place des règles anti-fraude (même contrat d'API)	B / M